

1. What If There Were No Operating System?

Without an operating system, a user would find it very difficult to use a computer because they would have to directly communicate with hardware.

Problems Without an Operating System

Without an OS:

1. No easy way to run programs.
2. No proper file management.
3. No easy communication with hardware devices.
4. No memory management.
5. No multitasking.
6. No user interface.
7. No security or user access control.

Without OS vs With OS

Feature	Without Operating System	With Operating System
File Management	User cannot easily create or organize files	OS manages files and folders
Program Execution	Difficult to run applications	OS loads and executes programs
Memory Management	Programs must manage memory directly	OS allocates and manages memory
Hardware Management	User must communicate directly with hardware	OS controls keyboard, mouse, printer, disk, etc.
Multitasking	Difficult to run multiple programs	Multiple programs can run together
Security	No user authentication or permissions	OS provides users, passwords, and permissions
User Interface	No simple interface	CLI or GUI is available

Linux Demonstration of OS Features

1: File Management

```
root@DESKTOP-E4HFS1K: ~/linux_demo
root@DESKTOP-E4HFS1K:~# mkdir linux_demo
root@DESKTOP-E4HFS1K:~# cd linux_demo/
root@DESKTOP-E4HFS1K:~/linux_demo# touch linux_text.text
root@DESKTOP-E4HFS1K:~/linux_demo# ls -l linux_text.text
-rw-r--r-- 1 root root 0 Aug 13 14:42 linux_text.text
root@DESKTOP-E4HFS1K:~/linux_demo#
```

2: Process Management

```
root@DESKTOP-E4HFS1K: ~
root@DESKTOP-E4HFS1K:~# pgrep bash
10
root@DESKTOP-E4HFS1K:~# ps -p 10
  PID TTY          TIME CMD
   10 tty1      00:00:00 bash
root@DESKTOP-E4HFS1K:~#
```

3: User and Security Management

```
root@DESKTOP-E4HFS1K:~/linux_demo# whoami
root
root@DESKTOP-E4HFS1K:~/linux_demo# id
uid=0(root) gid=0(root) groups=0(root)
root@DESKTOP-E4HFS1K:~/linux_demo# ls -l
total 0
-rw-r--r-- 1 root root 0 Aug 13 14:42 linux_text.text
root@DESKTOP-E4HFS1K:~/linux_demo#
```

2. Cybersecurity Laboratory – Linux OS Selection Guide

Linux Distribution	Main Purpose	Suitable Role	Important Features
Kali Linux	Penetration Testing	Ethical Hacker / Pen Tester	Security testing tools
Parrot OS	Security and Privacy	Security Researcher	Security tools and privacy features
Ubuntu	General Linux and Server Security	System Administrator	Stable and beginner-friendly
CAINE	Digital Forensics	Digital Forensic Investigator	Forensics and investigation tools
Security Onion	Security Monitoring	SOC Analyst	Network monitoring and threat detection
Tails	Privacy and Anonymity	Privacy Researcher	Privacy-focused environment

OS Selection Guide

1. Penetration Testing → Kali Linux

Recommended for: Ethical hackers and penetration testers.

Features:

- Nmap
- Wireshark
- Burp Suite
- Metasploit Framework
- Many security testing tools

Example use:

A penetration tester can use Kali Linux to scan and test systems in an authorized lab environment.

2. Privacy and Security → Parrot OS

Recommended for: Security researchers and privacy-focused users.

Features and Tools:

- **AnonSurf** – routes applications through Tor for privacy
- **Tor Browser** – privacy-focused web browsing
- **Nmap** – network discovery and security auditing
- **Wireshark** – network traffic analysis
- **Burp Suite** – web application security testing
- **Metasploit Framework** – penetration testing

Example use:

A security researcher can use Parrot OS with Nmap and Wireshark to study network security in an authorized laboratory.

3. Digital Forensics → CAINE Linux

Recommended for: Digital forensic investigators.

Features and Tools:

- **Autopsy** – digital forensic investigation and disk analysis
- **The Sleuth Kit (TSK)** – filesystem and disk-image analysis
- **Guymager** – forensic disk imaging
- **Foremost** – file recovery and data carving
- **Binwalk** – analysis and extraction of files from binary data
- **TestDisk** – partition and file recovery

Example use:

An investigator can use **Autopsy** and **The Sleuth Kit** to examine a forensic disk image and investigate files and filesystem activity.

4. Security Monitoring → Security Onion

Recommended for: SOC analysts and security monitoring teams.

Features and Tools:

- **Suricata** – network intrusion detection and prevention
- **Zeek** – network security monitoring and traffic analysis
- **Elastic Agent** – collection of endpoint and system data
- **Splunk** – security monitoring and endpoint detection
- **CyberChef** – data analysis and encoding/decoding
- **Kibana** – visualization and analysis of collected security data

Example use:

5. Privacy and Anonymity → Tails

Recommended for: Privacy-focused users, journalists, researchers, and users who need a privacy-oriented environment.

Features and Tools:

- **Tor Browser** – provides privacy-focused web browsing through the Tor network.
- **OnionShare** – allows files to be shared using the Tor network.
- **Thunderbird** – email client available in Tails.
- **KeePassXC** – password manager for securely storing passwords.
- **MAT2 (Metadata Anonymisation Toolkit)** – helps remove metadata from supported files.
- **Tails Greeter** – provides configuration options when starting the Tails system.

Example use:

A privacy-focused user can use **Tails with Tor Browser** to browse the Internet through the Tor network while reducing traces left on the computer when Tails is used as intended.

3.Exit Status in Linux

After executing a command, Linux returns an exit status.

The exit status tells us whether the command was successful or unsuccessful.

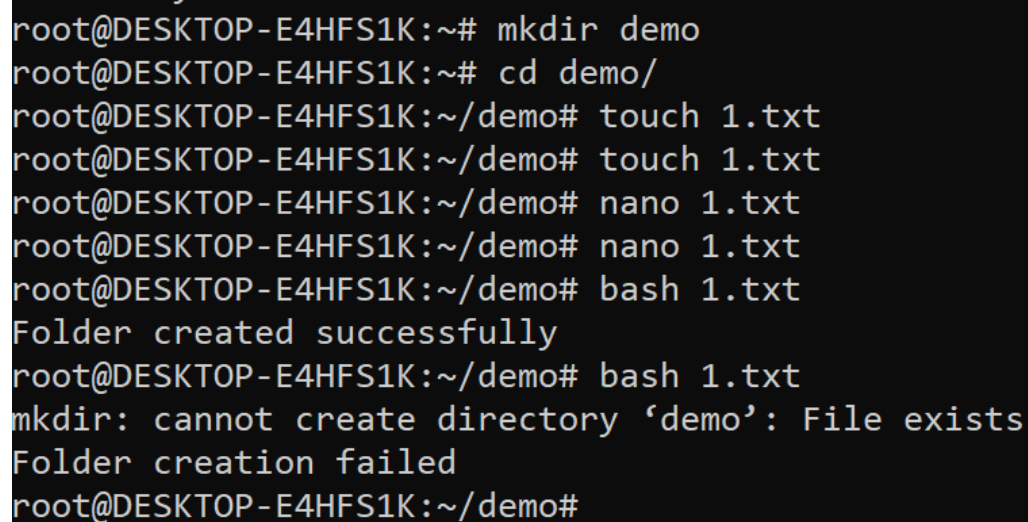
Command	Result	Exit Status	Meaning
pwd	Displays /root	0	Successful
ls /etc	Displays /etc contents	0	Successful
ls /does_not_exist	Displays “No such file or directory”	2	Path could not be accessed
false	Command returns failure	1	Unsuccessful
command_that_does_not_exist	Displays “command not found”	127	Command could not be found
mkdir Linux_test	Creates a directory	0	Successful

1: Successful Command AND Unsuccessful Command

```
root@DESKTOP-E4HFS1K:~# ls /does_not_exist
ls: cannot access '/does_not_exist': No such file or directory
root@DESKTOP-E4HFS1K:~# echo $?
2
root@DESKTOP-E4HFS1K:~# pwd
/root
root@DESKTOP-E4HFS1K:~# echo $?
0
root@DESKTOP-E4HFS1K:~# false
root@DESKTOP-E4HFS1K:~# echo $?
1
root@DESKTOP-E4HFS1K:~# command_that_does_not_exist
command_that_does_not_exist: command not found
root@DESKTOP-E4HFS1K:~# echo $?
127
root@DESKTOP-E4HFS1K:~# mkdir linux_test
root@DESKTOP-E4HFS1K:~# echo $?
0
root@DESKTOP-E4HFS1K:~#
```

Script Learn From an Exit Status

```
mkdir demo  
if [ $? -eq 0 ]  
then  
    echo "Folder created successfully"  
else  
    echo "Folder creation failed"  
fi
```



```
root@DESKTOP-E4HFS1K:~# mkdir demo  
root@DESKTOP-E4HFS1K:~# cd demo/  
root@DESKTOP-E4HFS1K:~/demo# touch 1.txt  
root@DESKTOP-E4HFS1K:~/demo# touch 1.txt  
root@DESKTOP-E4HFS1K:~/demo# nano 1.txt  
root@DESKTOP-E4HFS1K:~/demo# nano 1.txt  
root@DESKTOP-E4HFS1K:~/demo# bash 1.txt  
Folder created successfully  
root@DESKTOP-E4HFS1K:~/demo# bash 1.txt  
mkdir: cannot create directory 'demo': File exists  
Folder creation failed  
root@DESKTOP-E4HFS1K:~/demo#
```